D.S KOTHARI CENTRE FOR RESEARCH AND INNOVATION

DEPARTMENT OF PHYSICS

MIRANDA HOUSE, UNIVERSITY OF DELHI

Analysing Baryonic Acoustic Oscillations with MCMC: Insights from DESI data on tracers

By

Bhumika, B.Sc(H) Physics, Miranda House, DU
Jasleen Walia, B.Sc(H) Physics, Sri Venkateshwara College, DU
Km Vandana Sharma, B.Sc(H) Physics, Miranda House, DU

Mentors

Dr. Abha Dev Habib
Dr. Nisha Rani

Acknowledgement

We express our immense gratitude to the DSKC Summer Internship Programme 2024, Miranda House, coordinated by Prof. Monika Tomar, for providing us with this invaluable opportunity to come together as young science enthusiasts and engage in collaborative projects this summer. Our thanks also extend to DSKC for ensuring workspace and accommodation facilities, enabling us to work efficiently as a group.

Our deepest appreciation goes to Dr. Abha Dev Habib and Dr. Nisha Rani for their continuous guidance throughout the project. Their valuable insights and constructive criticisms were instrumental in our progress. We also thank them for organizing and teaching the lecture series on various statistical tools, which greatly enhanced our understanding.

We are also grateful to Dr. Akshay Rana for his enlightening guest lecture on the Markov Chain Monte Carlo method, a key component of our research paper study.

Finally, we extend our heartfelt thanks to all the students of the DSKC Cosmology group. Their collective effort in creating a fun and enthusiastic environment for learning, working, and bonding made this experience truly memorable.

This summer has been a profound learning journey, allowing us to develop valuable skills. We are thankful to everyone involved in making this happen.

Principal

Introduction

During this internship programmme, we have learnt some computational techniques which include Monte Carlo Methods and Markov Chain Monte Carlo.

We study the interesting paper “Does DESI Confirm ΛCDM?" by E. O Colg´ ain, M. G. Dainotti, Capozziello, Pourojaghi, M. M. Sheikh-Jabbari, and D. Stojkovic [1]. In this paper, a discrepancy was discovered in DESI Luminous Red Galaxy (LRG) data, indicating a slightly higher matter density ($\Omega_m \approx 0.66$). We referenced another paper "DESI 2024 VI: Cosmological Constraints from the Measurements of Baryon Acoustic Oscillations" [2] which provides measurements of Baryon Acoustic Oscillations (BAO) of galaxies, quasars, and Lyman-α forest tracer from the first year of observations by the Dark Energy Spectroscopic Instrument (DESI).

We applied the techniques learnt (Appendix 1) to reproduce the results of Table 1 of [1]. Using the Markov Chain Monte Carlo technique, we analysed the data in Table 1 of [2] to determine the values of $H_0 r_d$ and Ω_m from a cumulative χ^2 analysis.

The contents of this project report begin with a comprehensive overview of cosmology in The ΛCDM Model and Cosmic Evolution. This section introduces to the evolution of the universe, the Friedmann Equation, and the ΛCDM model, laying a foundation for understanding the cosmic expansion. The second section, Baryon Acoustic Oscillations (BAO), delves into the formation and significance of BAO, focusing on important concepts like the sound horizon, comoving distances, and their use as a standard ruler in cosmology. The third section, Dark Energy Spectroscopic Instrument (DESI) introduces to the DESI telescope and explores various galaxy and quasar tracers. Section four, Data and Methodology, covers chi-square analysis and Markov Chain Monte Carlo techniques that are applied on the data of Table 1 of [2]. The final sections cover Conclusions and Future Projects, followed by References for further reading. The appendices provide additional resources, with Appendix I detailing statistical tools learnt during the internship and Appendix II presenting relevant Python codes to assist with the analyses.

Contents

1. The ΛCDM Model and Cosmic Evolution

1.1 Evolution of Universe: From Big Bang to Dark Energy Epoch

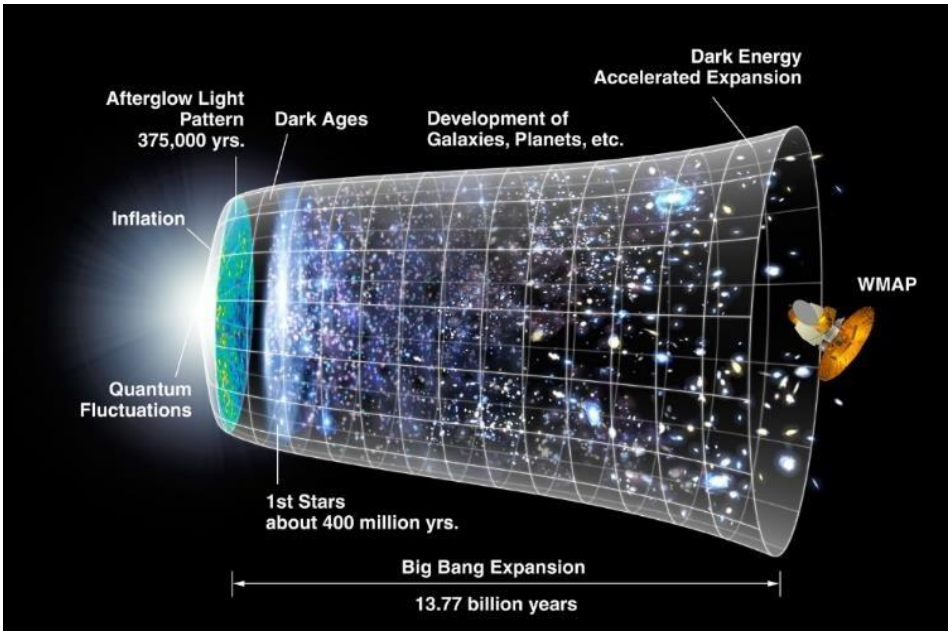

Fig. 1 Timeline of the universe from the Big Bang to dark energy-driven expansion[5]

The universe began with big bang (13.8 billion years ago) at t=0, and an enormous explosion. It then expanded exponentially from an initial singularity. During the Planck epoch ($t = 10^{-43}$ s) when the universe's density exceeded a critical density and the quantum gravitational effect was dominant, leading to the separation of fundamental forces ($10^{-43} \leq t \leq 10^{-36}$s). Inflation occurred ($t < 10^{-32}$s) that smoothed out any irregularities and set the stage for the formation of cosmic structures. At t = 10^{-12}s, quarks began to exist. At t = 10^{-11} matter began to form in the form of baryons (protons and neutrons) resulting in matter–antimatter asymmetry. In $10^{-6} < t < 1$ s hadrons dominated the universe's density leading to the formation of photons, then neutrinos decoupled and began to propagate freely.

After the First second, for about 13.77 billion years, the universe has undergone many events. At 10 s < t < 377,000 years photons dominated the universe's energy density which led to the formation of light elements such as hydrogen and helium. At t = 37,000 years matter's density surpassed that of radiation, driving the universe's evolution. Ionized particles combined to form neutral atoms, making the universe transparent and releasing the Cosmic Microwave Background (CMB). For 380,000 years to 1 Gyr the universe was completely dark and stars had not yet formed. Stars,

dwarf galaxies and quasars began to form, with population III stars (These are hypothetical population of stars formed shortly after Big Bang. These are massive, extremely hot and composed of astronomical metals with no virtual heavy element.) appearing around 700 Myr. Radiation from these population III stars ionised the neutral hydrogen leading to the formation of plasma of protons and electrons. At t > 1Gyr Larger galaxies and galaxy clusters formed that contain population II stars followed by population I stars. For t > 9.8 Gyr the universe was in the Dark energy epoch, where dark energy drives the accelerated expansion of the universe. (These are hypothetical population of stars formed shortly after Big Bang . These are massive, extremely hot and composed of astronomical metals with no virtual heavy element.) [5]

Our understanding of how the Universe evolves can be contained in a Cosmological model. Such models are specified by a number of parameters, such as the densities of radiation and matter, the Universe's curvature, and the Hubble parameter and few more. These parameters are measured observationally to determine which cosmological model best describes the Universe, with different experiments being able to constrain different parameters.

1.2 The Friedmann Equation and Cosmological Expansion

The basis of understanding how the expansion of the universe is accelerating lies in the Einstein equations. One of the exact solutions known of Einstein field equations is the Friedmann–Lemaitre–Robertson–Walker (FLRW) solution, describing an expanding universe. Alexander Friedmann derived two equations starting from Einstein's equations and FLRW solution which are known as Friedmann Equations.

The Friedmann equation is :

$$H^2 - \frac{8\pi G\boldsymbol{\rho}}{3} = \frac{kc^2}{a^2} \quad (1)$$

Expansion – density = curvature

In equation (1), H is the "Hubble Parameter" which determines the expansion rate of the universe. The present-day value of expansion of universe is given by "Hubble Constant H_0. ρ is the average density of matter and energy in the universe, a is the scale factor, and k is a number that describes the overall curvature of the universe. Thus, the expansion of the universe is related to its contents as well as the curvature of the space. There is a special value of density $\rho_{critical}$ that causes the universe to have zero curvature.

$$\rho_{critical} = \frac{3H^2}{8\pi G} \quad (2)$$

In equation (1) if $\rho = \rho_{critical}$, we say that such a universe is flat.

1.3 The ΛCDM Model

The ΛCDM model describes a universe with two key features: the energy density of dark matter is dominated by cold dark matter (CDM), and the total energy density includes a significant contribution from the cosmological constant Λ, which represents dark energy. The ΛCDM model posits that the universe consists of approximately 5% baryonic matter, 25% cold dark matter, and 70% dark energy in the form of Einstein's cosmological constant (Λ). Smaller contributions come from massive neutrinos and radiation. This model is currently the best fit for describing a flat universe with zero curvature, explaining the universe's accelerated expansion.

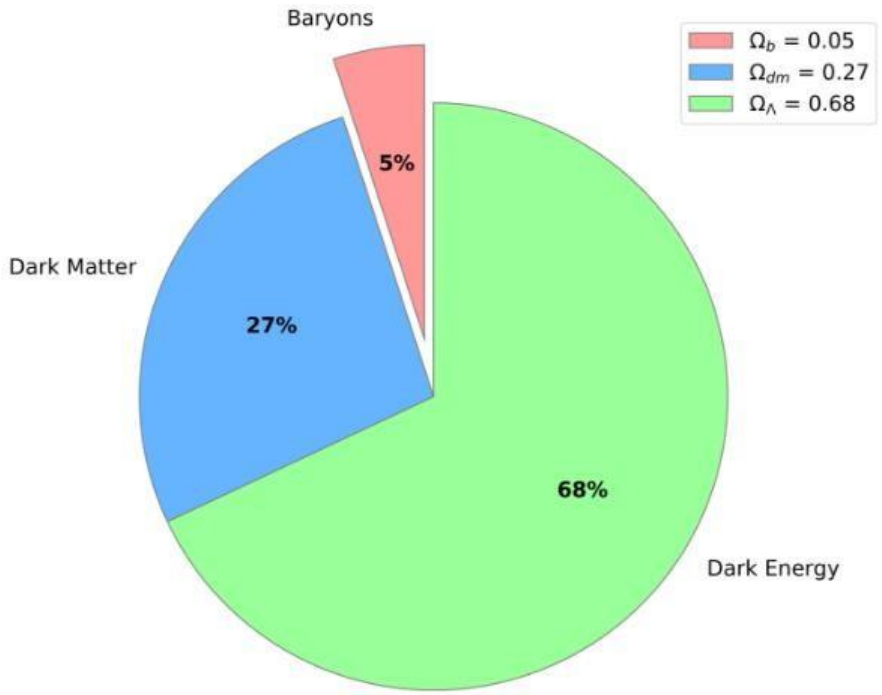

Fig. 2 ΛCDM Universe: 68% dark energy, 27% dark matter, 5% familiar matter [5]

2. Baryon Acoustic Oscillations (BAO)

Baryon Acoustic Oscillations (BAO) refer to periodic fluctuations in the density of visible baryonic matter (normal matter) in the universe. These oscillations are imprints from the early universe and serve as a standard ruler for measuring cosmic distances and understanding the expansion history of the universe.

2.1 Formation of BAO

In the early universe, photons and baryons (protons and neutrons) were tightly coupled in hot, dense plasma. Acoustic waves or sound waves, travelled through this plasma due to the pressure exerted by the photons against the baryons. These sound waves set up a regular pattern of density variations, known as baryon acoustic oscillations. As the universe expanded and cooled, these oscillations were frozen in the matter distribution as the plasma decoupled and the universe transitioned to a state where matter was no longer tightly coupled to radiation.

The BAO feature is thus a result of this sound wave propagation, leaving a characteristic scale in the matter distribution that we can observe today.

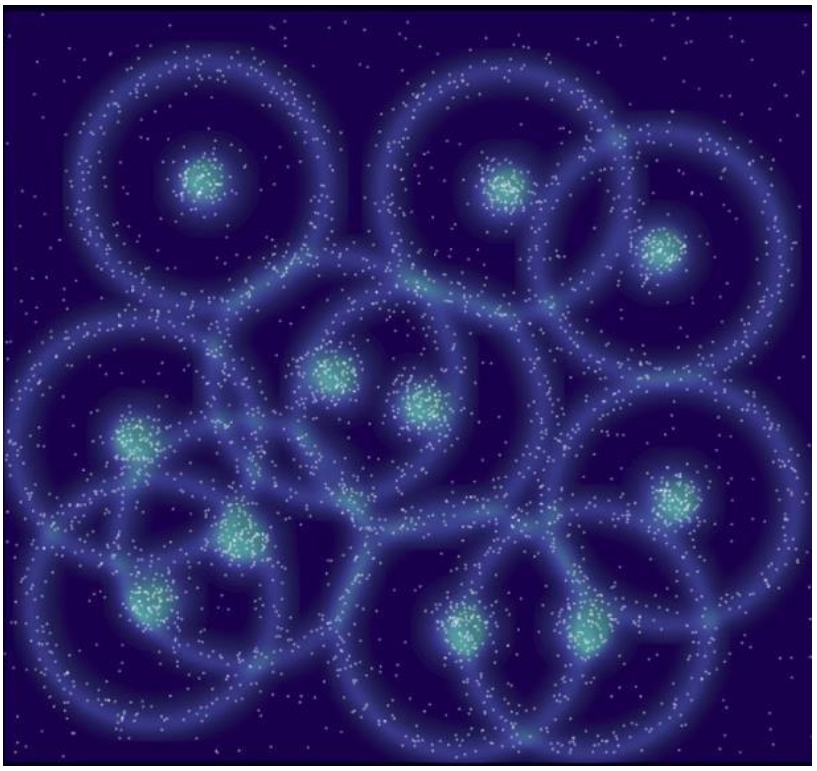

Fig. 3 Sound waves imprint the BAO signature on matter distribution [4]

2.1.1 Sound Horizon (r_d)

The sound horizon at the baryon drag epoch, when photons decouple the baryons and travel freely through space, denoted as r_d is the maximum distance that sound waves could travel in the early universe before the baryons decoupled from the photons. It serves as a cosmic standard ruler for measuring distances in the universe. The expression for r_d is –

$$r_d = \int_{z_d}^{\infty} \frac{c_s(z)}{H(z)} dz \quad (3)$$

where $c_s(z)$ is the speed of sound prior to recombination and is given by,

$$c_s(z) = \frac{c}{\sqrt{3}} \left(1 + \frac{3\rho_\beta}{\rho_\gamma}\right)^{\frac{-1}{2}} \quad (4)$$

where ρ_B and ρ_γ are the baryon and radiation energy densities, respectively.

2.1.2 Transverse Comoving Distance ($D_M(z)$)

This is the distance measured in a direction perpendicular to the line of sight at a given redshift (z). It is related to (r_d) through the observed angular separation of galaxies:

$$D_M(z) = \frac{c}{H_0} \int_0^z \frac{dz'}{E(z')} \quad (5)$$

where $E(z)=\sqrt{(\Omega_m(1+z)^3 + 1 - \Omega_m)}$

Ω_m = matter density

z = redshift

2.1.3. Hubble Distance ($D_H(z)$)

This measures the distance along the line of sight and is inversely proportional to the Hubble parameter at redshift z:

$$D_H(z) = \frac{c}{H(z)} \quad (6)$$

2.1.4. Volume Distance ($D_v(z)$)

This combines the transverse and line-of-sight distances into a single quantity that averages over both dimensions:

$$D_V(z) = \left(\frac{D_V(z)z^2}{z}\right)^{\frac{1}{3}} \quad (7)$$

2.2 BAO as a Standard Ruler

The BAO feature manifests as a distinct peak in the galaxy correlation function or as oscillations in the galaxy power spectrum. This peak allows astronomers to measure cosmic distances accurately because the comoving scale of r_d is set by early universe physics and can be calibrated with high precision using other measurements, such as those from the Cosmic Microwave Background (CMB). By measuring the separation of this peak in the galaxy distribution, scientists can determine distances and the expansion rate of the universe, providing critical insights into cosmological parameters and the dynamics of dark energy.

3. DESI

The Dark Energy Spectroscopic Instrument (DESI) is a cutting-edge astronomical survey designed to probe dark energy and the large-scale structure of the universe. Mounted on the Nicholas U. Mayall Telescope at Kitt Peak National Observatory, DESI employs ten advanced spectrographs to measure the redshifts of various celestial objects, including galaxies, quasars, and the Lyman-α Forest. The data collection process involves precise fibre optics and robotic positioners that direct light from the observed objects into the spectrographs. Each observation field, known as a “tile,” is covered by up to 5000 fibres, which are positioned to capture light from targeted celestial sources. This setup allows DESI to gather detailed spectra, which are then used to determine the redshifts of the objects and analyse their distribution across the sky. The resulting catalogues provide critical insights into the expansion rate of the universe and the influence of dark energy, making DESI a vital tool for cosmological research.

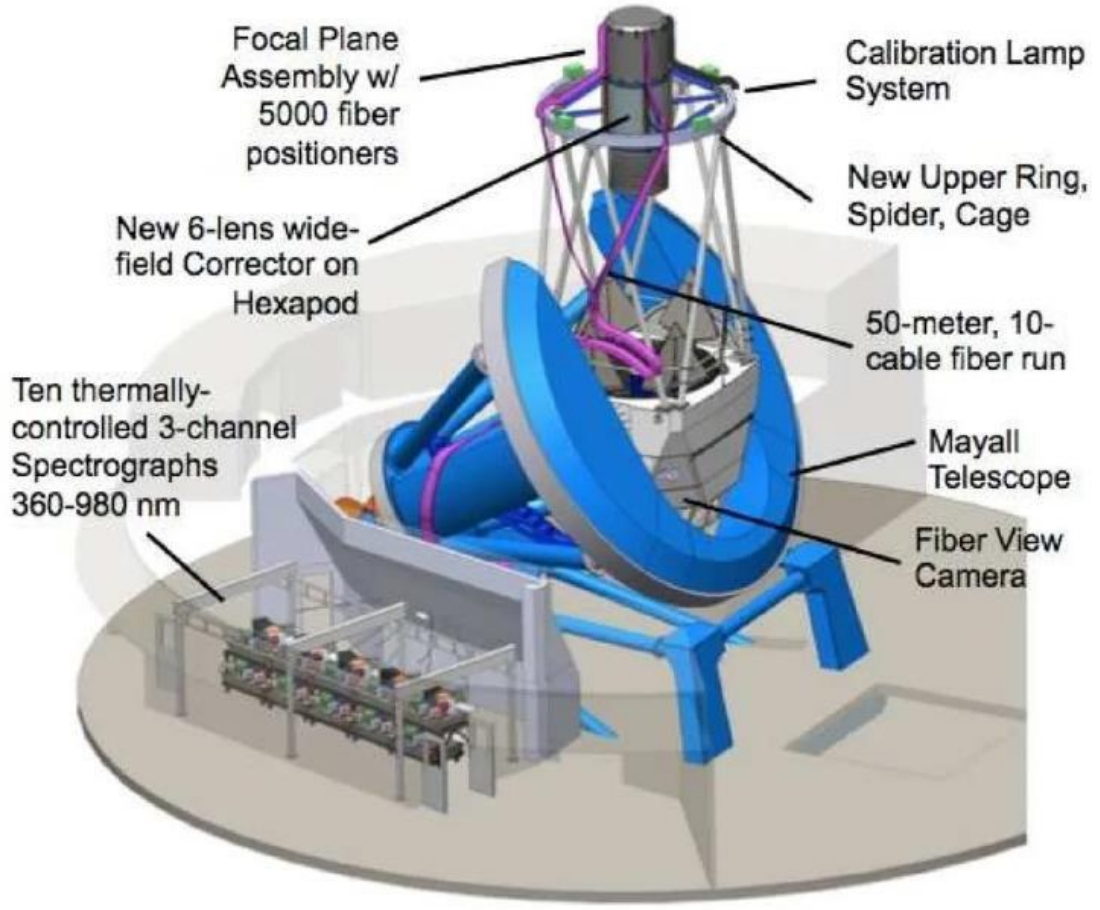

Fig.4 The setup of the Dark Energy Spectroscopic Instrument (DESI). New detectors and computer controlled fibre optics have been installed on the Mayall Telescope making it the most sensitive instrument for studying Dark Energy. (Credit: Spiedigitallibrary)

The use of galaxies as tracers of large-scale structures has been crucial in cosmology, offering important complementary information to that from Type Ia supernovae (SN Ia) and the Cosmic Microwave Background (CMB). Over the past fifty years, galaxy clustering has evolved significantly, utilizing larger and more detailed galaxy catalogs. It has become a key cosmological measurement, directly linking observations to the properties of dark matter and dark energy.

3.1 TRACERS

3.1.1 Bright Galaxy Sample (BGS)

The Bright Galaxy Sample (BGS) targets galaxies within $0.1 < z < 0.4$, using r-magnitude cuts to ensure uniform density across the photometric samples and the GAIA catalogue to remove point-like sources. This results in 854 targets per square

degree with high priority for bright time observations. The sample is flux-limited and shows strong redshift evolution. To create a more homogeneous sample, a cut was applied to maintain a constant comoving number density of 5×10^{-4} h^3 Mpc^{-3} using k-corrected r-band absolute magnitudes and redshift-dependent corrections. The BGS number density is comparable to LRGs at z = 0.4, minimizing shot noise in BAO statistical uncertainty.

3.1.2 The Luminous Red Galaxy (LRG)

The Luminous Red Galaxy Sample (LRG) targets galaxies within $0.4 < z < 0.8$ using photometry from Wide – field Infrared Survey Explorer (WISE) to select red objects with nearly constant comoving number density of $5 \times 10^{-4} h^3 Mpc^{-3}$. Target sample density is 624 targets per square degree. The lower redshift bound was chosen to separate the sample from BGS, as most low – redshift LRG targets are also BGS targets, while the upper bound was chosen to ensure a minimum density of $10^{-4} h^3 Mpc^{-3}$. The sample is further divided into 3 disjoint redshift ranges, $0.4 < z < 0.6$, $0.6 < z < 0.8$, and $0.8 < z < 1.1$; referred as LRG1, LRG2 and LRG3 respectively.

3.1.3 The Emission Line Galaxy (ELG)

The Emission Line Galaxy (ELG) targets galaxies within $1.1 < z < 1.6$. Targets are selected using colour cuts in the (g – r) vs (r – z) plane to prioritise objects with [OII] emission in the desired redshift range. It utilizes photometry from WISE to select red objects with a constant comoving number density of 5×10^{-4} h^3 Mpc^{-3}. Lower bound aligns with the high–redshift LRG sample. The upper bound ensures the [OII] doublet remains within spectrograph wavelength coverage.

3.1.4 The Quasar (QSO)

The Quasar Sample (QSO) targets galaxies within $0.8 < z < 2.1$. The selection relies on identifying a flux excess in the near–infrared using WISE W1 and W2 magnitudes compared to Legacy Imaging Surveys optical grz magnitudes. It uses a Random Forest (RF) algorithm with grzW1W2 magnitudes to select quasars, targeting a sample density of 310 $(deg)^2$. Quasar targets are assigned fibres with maximum priority.

3.1.5 The Lyman-α Forest (Lyα)

The Lyman-α Forest Sample (Lyα) targets galaxies within $1.77 < z < 4.16$. It is obtained from a combined analysis of correlations of three different datasets. This first dataset consists of Quasars in the redshift range $1.77 < z < 4.16$. In addition, the

Lyα at z > 2.1 constitutes an alternative tracer of matter density fluctuations. The two datasets consist of two Lyα forest datasets, consisting of fluctuations in two different rest-frame wavelength regions of the background quasar spectra. Quasars identified as having z > 2.1 are prioritized for up to four observations to increase the signal-to-noise ratio of the spectra.

4. Data and Methodology

We employ MCMC to determine the 68% confidence intervals for ΛCDM parameters (Ω_m and $H_0 r_d$) from BAO data. The observables include LRG, Emission Line Galaxies (ELG) and the Lyman α forest (Lyα QSO).

Tracer	Redshift	N_{tracer}	z_{eff}	$D_M(z)/r_d$	$D_H(z)/r_d$	$D_V(z)/r_d$
BGS	0.1 – 0.4	300,017	0.295	-	-	7.93 ± 0.15
LRG1	0.4 – 0.6	506,905	0.510	13.62 ± 0.25	20.98 ± 0.61	−0.445
LRG2	0.6 – 0.8	771,875	0.706	16.85 ± 0.32	20.08 ± 0.60	-0.420
LRG3+ELG1	0.8 – 1.1	1,876,164	0.930	21.71 ± 0.28	17.88 ± 0.35	-0.389
ELG2	1.1 – 1.6	1,415,687	1.317	27.79 ± 0.69	13.82 ± 0.42	-0.444
QSO	0.8 – 2.1	856,652	1.491	-	-	26.07 ± 0.67
Lya QSO	1.77 – 4.16	709,565	2.330	39.71 ± 0.94	8.52 ± 0.17	-0.477

Table I. For each tracer and redshift range, N_{tracer} is the total number of objects within the reshift range [2]

Denoting d_1 for $D_M(z)/r_d$, d_2 for $D_H(z)/r_d$ and d_3 for $D_V(z)/r_d$ for each measurement in table 1 as follows [6]:

$$\chi^2_{D_M} = \sum_{i=1}^{5} \frac{(d_1^{obs}(z_i) - d_1^{th}(z_i))^2}{(\sigma_{d_1}^{obs})^2} \quad (8)$$

$$\chi^2_{D_H} = \sum_{i=1}^{5} \frac{(d_2^{obs}(z_i) - d_2^{th}(z_i))^2}{(\sigma_{d_2}^{obs})^2} \quad (9)$$

$$\chi^2_{D_V} = \sum_{j=1}^{2} \frac{(d_3^{obs}(z_j) - d_3^{th}(z_j))^2}{(\sigma_{d_1}^{obs})^2} \quad (10)$$

Here, d^{obs} stands for the observed distance value as presented in Table I, while d^{th} represents the calculated theoretical value for the models being examined.

The cumulative chi-squared expression for the BAO measurements, represented χ^2_{final} as BAO, is formulated as follows:

$$\chi^2_{final} = \chi^2_{D_M} + \chi^2_{D_H} + \chi^2_{D_V} \quad (11)$$

5. Results and Discussion

Maximizing $-\chi^2_{final}$ (11) which is equal to logarithm of Likelihood, we find the corresponding best fit values of Ω_m and $H_0 r_d$. Without external data or assumptions, it is impossible to separate H_0 from r_d, so we quote confidence intervals for the combination $H_0 r_d$ alongside Ω_m (**Fig. 5**). We applied a Gaussian prior on $r_d \sim N(147.46, 0.28)$ Mpc determined using constraints from Lemos & Lewis (2023) [3].

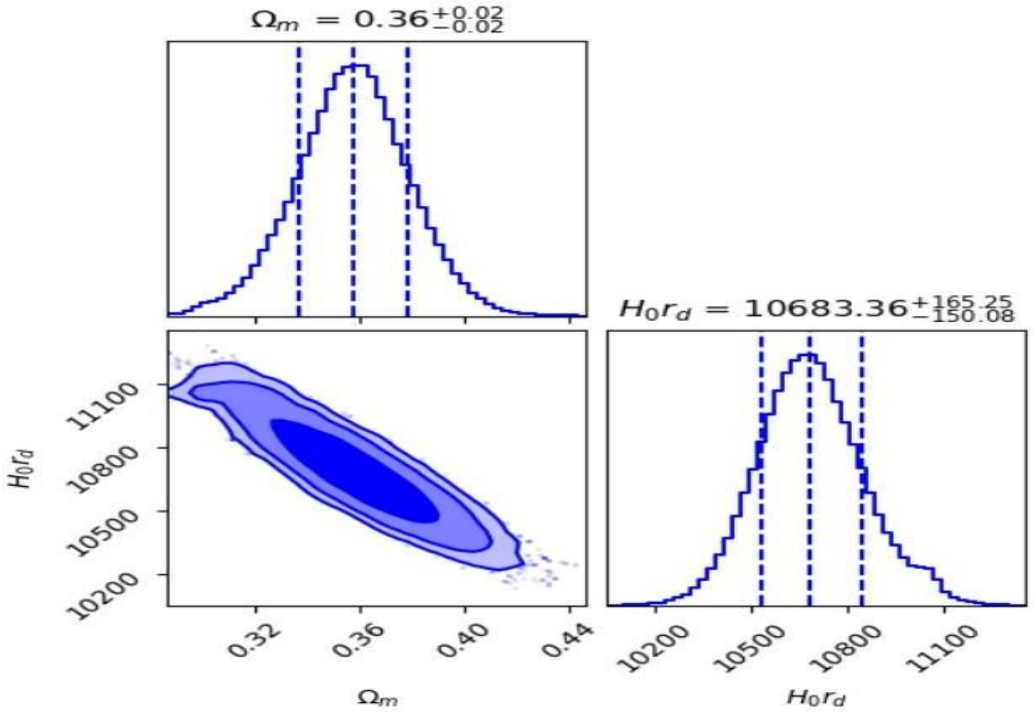

Fig. 5 1σ confidence intervals from DESI BAO data [Appendix II -7]

From **Fig. 5,** the best fit values of the Ω_m is $0.36^{+0.02}_{-0.02}$ and $H_0 r_d$ is $10.68^{+1.65}_{-1.50}$ (100 km/s).

The surface plot of log(posterior) is –

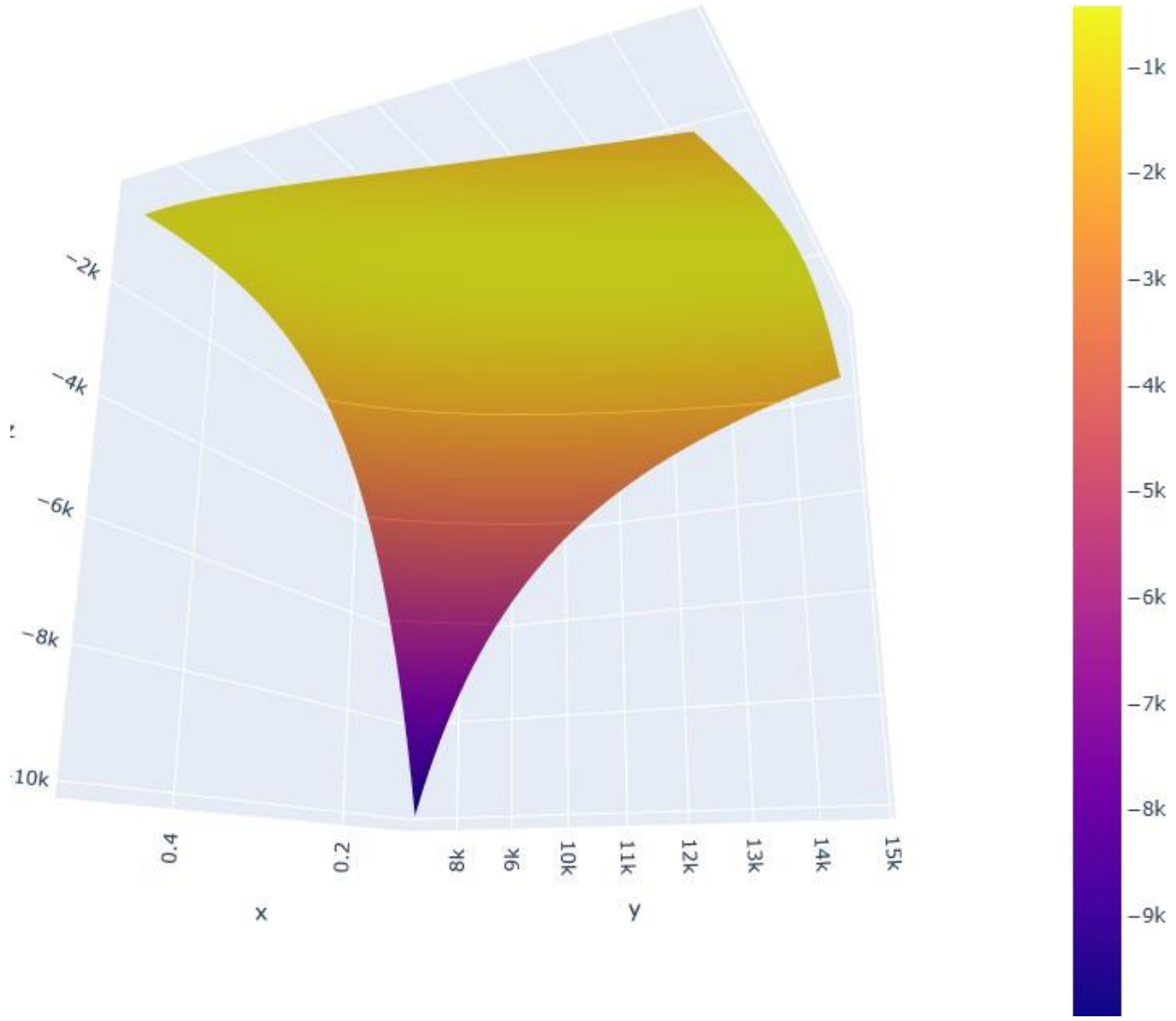

Fig. 6 Surface plot of $-\chi^2_{final}$ (11). This plot displays the set of three dimensional data of Ω_m , H_0r_d and log Posterior as a mesh surface, With the help of the colour contrast we can see that log posterior has only one maxima, which occur at the best fit values of Ω_m and H_0r_d [Appendix II – 6].

For each tracer given in Table 1 of [2], we quote value of χ^2_{min}, Ω_m and H_0r_d and extend the results of table 1 of [1].

Tracer	z_{eff}	H_0r_d (100 km/s)	Ω_m
LRG1	0.51	$88.56^{+4.69}_{-4.33}$	$0.66^{+0.16}_{-0.15}$
LRG2	0.71	$108^{+4.97}_{-5.00}$	$0.23^{+0.07}_{-0.06}$
LRG + ELG	0.93	$102.42^{+3.34}_{-3.38}$	$0.27^{+0.05}_{-0.04}$
ELG	1.32	$98.18^{+6.11}_{-6.66}$	$0.34^{+0.08}_{-0.07}$
Lyα QSO	2.33	$93.34^{+6.27}_{-6.24}$	$0.37^{+0.07}_{-0.06}$

TABLE II. 1σ confidence intervals from DESI anisotropic BAO constraints at effective redshift, z_{eff} [Appendix II – 8]

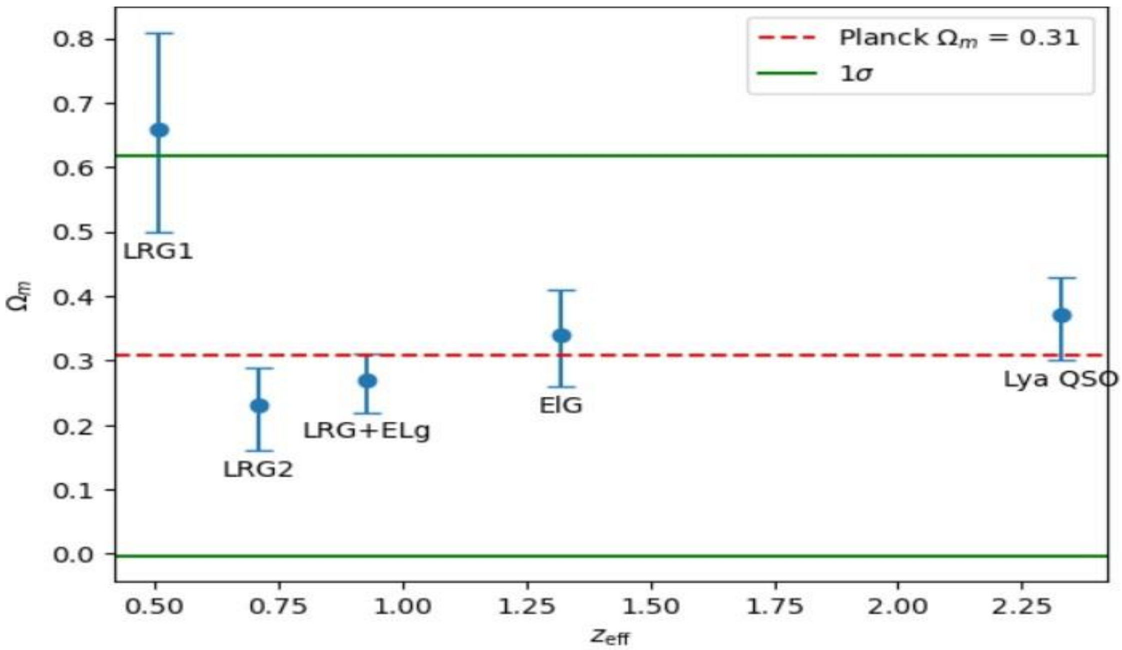

Fig. 7. 1σ confidence intervals for Ω_m versus z_{eff}. [Appendix II – 9]

Based on the observations from Table II, as we move towards higher redshifts, the values of Ω_m increase, while the values of H_0r_d decrease. If we neglect the DESI LRG constrain at effective redshift z_{eff} = 0.51, the data shows excellent agreement with the corresponding Planck value $\Omega_m = 0.315 \pm 0.007$ at the ~ 2.1 σ level. Furthermore, when we performed a cumulative chi-squared analysis, we found a slightly higher value of Ω_m, agrees with the Planck value of Ω_m.

5. Conclusion and Future Work

We have reproduced the results of paper named “Does DESI Confirm ΛCDM?" by E. O Colg´ ain, M. G. Dainotti, Capozziello, Pourojaghi, M. M. Sheikh-Jabbari, and D. Stojkovic[1] in which we have used DESI BAO data[2] to find out the matter density Ω_m and $H_0 r_d$. Our analysis reveals a significant discrepancy between the DESI LRG data at an effective redshift z_{eff} = 0.51 and the predictions of Planck ΛCDM cosmology. Specifically, this discrepancy translates into an unexpectedly large value of matter density parameter, $\Omega_m = 0.66^{+0.16}_{-0.15}$ which deviates from Planck’s ΛCDM value.

This anomaly suggests a preference for a time-varying dark energy equation of state in the $w_0 w_a$CDM. Furthermore, our redshift bin analysis indicates that the parameter Ω_m variations at the $\sim 2\sigma$ level with increasing effective redshift within the ΛCDM framework. We anticipate that future DESI data releases will reduce the statistical significance of this anomaly. However, we currently observe a potential trend of increasing Ω_m with effective redshift. This trend underscores the necessity to investigate why the DESI LRG data at an effective redshift z_{eff} = 0.51 appears as an outlier in the determination of Ω_m.

In the future, we can analyse this using a time-varying dark energy equation of state to determine if this anomaly persists in such models. Additionally, we can incorporate more refined data from DESI and other observational surveys to better understand the underlying causes and potential implications of this trend. This comprehensive approach may reveal whether the observed anomaly is a statistical fluke or indicative of new physics beyond the standard ΛCDM model.

References

[1]. E. O Colg´ ain, M. G. Dainotti, Capozziello, Pourojaghi, M. M. Sheikh-Jabbari, and D.Stojkovic, “Does DESI Confirm ΛCDM” [arXiv:2404.08633 [astro-ph.CO]]

[2]. A. G. Adame et al. [DESI], “DESI 2024 VI: Cosmological Constraints from the Measurements of Baryon Acoustic Oscillations,” [arXiv:2404.03002 [astro-ph.CO]]

[3]. Vikrant Yadav, “Measuring Hubble constant in an anisotropic extension of ΛCDM model”, Phys. Dark Univ. 42, 101365 (2023), [arXiv:2306.16135 [astro-ph.CO]]

[4].SciTech Daily, “Echoes of the Universe's Creation: Baryon Acoustic Oscillations”,[Echoes of the Universe's Creation: Baryon Acoustic Oscillations (youtube.com)]

[5]. Dr Vicky Scowcroft , “Relativistic Cosmology Part 2” Relativistic Cosmology Part 2 (vickyscowcroft.github.io]

[6]. R. Camilleri, The Dark Energy Survey Supernova Program: An updated measurement of the Hubble constant using the Inverse Distance Ladder, [arXiv:2406.05049 [astro-ph.CO]]

Appendix – I

Computational Techniques for Cosmological Analysis

This appendix provides a brief overview of the computational techniques learnt during this internship.

1. Monte Carlo Methods

Monte Carlo methods are a broad class of computational algorithms that rely on random sampling to solve problems or estimate properties of complex systems.

1.1 Estimation of pi

Here, we use "hit or miss" Monte Carlo method. We have a circle inscribes in a square. We throw random numbers in the square and count the random numbers that are falling inside the circle.

That is, $\frac{N}{N_s} = \frac{A_c}{A_s}$

N_c = No. of random numbers falling inside the circle

N_s = Total no. of random numbers

A_c = Area of circle $= \pi a^2$

A_s = Area of square $= (2a)^2$

Thus, equation (1) becomes –

$$\frac{\pi}{4} = \frac{N_c}{N_s}$$

$$\pi = 4 \times \frac{N_c}{N_s}$$

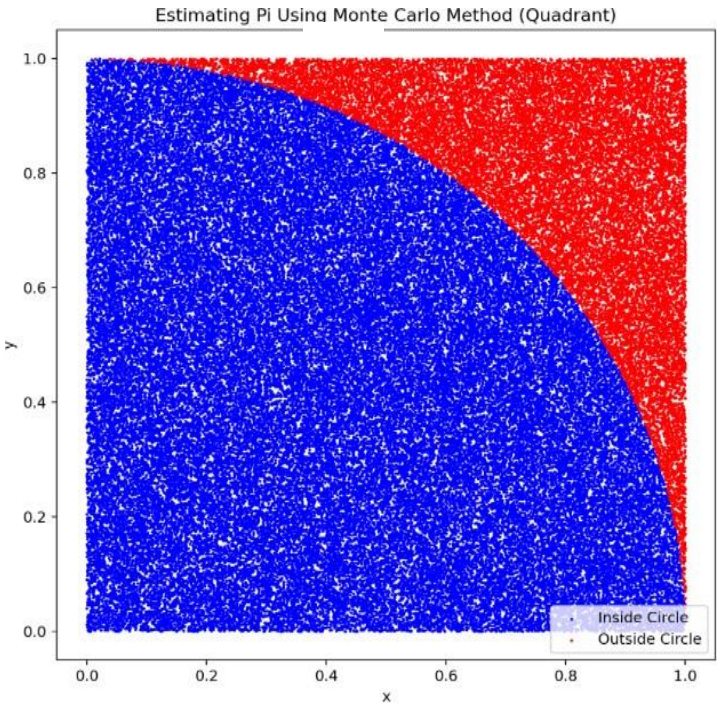

Fig. 8 Random numbers falling inside a square, Total no, of random numbers = 100000 and the value of π = 3.14356 [Appendix II – 1]

1.2 Estimation of the area under the curve:

The same technique which we were following to estimate the value of π can be used to find the area under the curve. To achieve this:

a) Find out the maxima of the curve in the region of the interest and check whether the function changes sign or not.
b) We draw a rectangle that'll have the height equal to the maximum value of the function and width equal to the length of the interval.
c) We will throw random numbers in the rectangle and count how many of then are falling in the area under the curve, then do the following mathematics -

A = Area of the rectangle
(b – a) = length of the interval
A_c = Area under the curve
N = Total number of points thrown

c = No. of points in the area under the curve

$$A = (b - a) \times ((b) - (a))$$

$$A_c = \int_a^b f(x)dx$$

$$\frac{A_c}{A} = \frac{c}{N}$$

$$A_c = \frac{c}{N} \times A$$

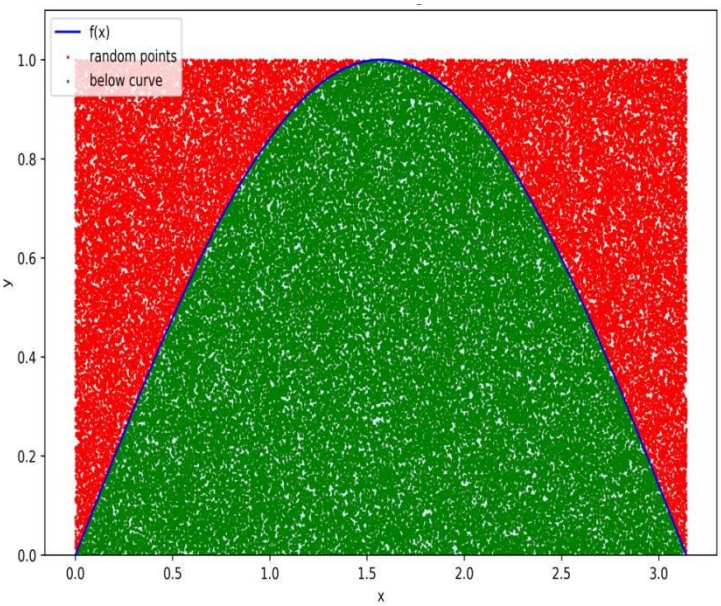

Fig. 9 Random numbers falling inside a rectangle, Total number of points = 5000 and Area under the curve, A_c = 1.9979 [Appendix II – 2]

2. Likelihood Analysis and Chi – Square fitting

2.1 Bayesian Approach

Traditional probability uses a frequency to try and estimate the probability of an occurrence and results in yes and no answers. Bayesian approach is aimed at finding a certain "confidence" in occurrence of a certain event (or parameter values in modelled events). In Bayesian probability, we begin with some initial understandings via data, and work our way towards a subjective understanding about the likeliness of an event. Thus Bayes' statistics aim to calculate the probability of one event (or parameter) based on the probabilities of occurrence of other events. Conditional probability is defined as the probability of a certain event or occurrence, given that another event has happened. It is a revised probability of an event B occurring given the knowledge that an event A has already occurred, and is denoted by P(B|A). When events A and B are independent (event B is not influenced by event A), the conditional probability of B, given a certain event A is just the probability of event B,
P(B). If the events are not independent, then the conditional probability is given by:

$$P(B \mid A) = \frac{P(A \cap B)}{P(A)} \tag{12}$$

Bayes Theorem provides us with knowledge to proceed with the under- standing of likelihood. The statement of Bayes Theorem states that a set of events H1, H2, ...Hk, associated with a sample space S, have all non-zero probability of occurrence and form a part of S. For any event D that occurs with the above events, Bayes Theorem says,

$$P(H_k \mid D) = \frac{P(H_k)P(D \mid H_k)}{\Sigma\, P(H_k)P(D \mid H_k)} \tag{13}$$

$$\text{i.e. Posterior} = \frac{Prior \times Likelihood}{evidence}$$

2.2 Maximum Likelihood Estimation

Likelihood function is a fundamental concept in statistical inference (2). It indicates how likely a particular population is to produce an observed sample. By maximizing the Likelihood function we can find the best fit parameters to our data.

2.3 Chi-square method:

Sometimes maximizing the likelihood function is difficult, here we define

$$\chi^2 = -2 \log_e(Likelihood) \tag{14}$$

Chi square test is used to test how likely the observed distribution of data fits with the expected distribution. We can minimize χ^2 to find the best fit parameters.

Formula:

$$\chi^2 = \sum \frac{(observed - theoretical)^2}{(error)\text{^}2} \tag{15}$$

By minimizing chi- square we can find the best fit value of parameters of a given model.

We have the following data:

z (redshift)	H(z) (Hubble parameter) km/sec/Mpc	σ_H
0.07	69	19.6
0.09	69	12
0.12	68.6	26.2
0.17	83	8
0.179	75	4
0.199	75	5
0.2	72.9	29.6
0.27	77	14
0.28	88.8	36.6
0.352	83	14
0.3802	83	13.5
0.4	95	17
0.4004	77	10.2
0.4247	87.1	11.2
0.4497	92.8	12.9
0.47	89	34
0.4783	80.9	9
0.48	97	62

0.5929	104	13
0.6797	92	8
0.7812	105	12
0.8754	125	17
0.88	90	40
0.9	117	23
1.037	154	20
1.3	168	17
1.363	160	33.6
1.43	177	18
1.53	140	14
1.75	202	40
1.965	186.5	50.4

Here we're are considering spatially flat ΛCDM model where $\Omega_m + \Omega_\Lambda = 1$ and we have the following equation of hubble parameter -

Theoretical model:

$H(z) = H_0 (\Omega_m (1+z)^3 + 1 - \Omega_m)^{0.5}$

a) Consider $H_0 = 67.8$

We can minimize χ^2 to find the best fit value of Ω_m.

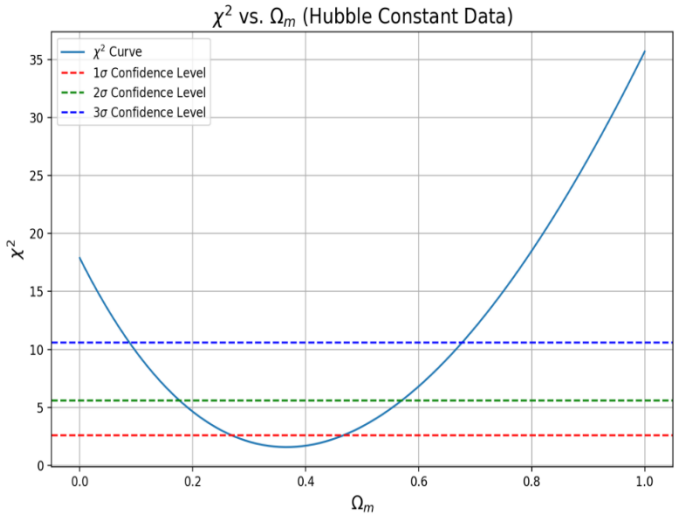

Fig. 10 One parameter fitting using chi – square analysis keeping the other parameter i.e. H_0 = constant.
[Appendix II – 3]

b) Considering both H_0 and Ω_m as free parameters, we can minimize to find χ^2 the best fit value of these parameters.
Minimum χ^2 value = 1.571114706973622
Optimal Ω_m = 0.34867383338134034
Optimal H_0 = 68.20667649584738

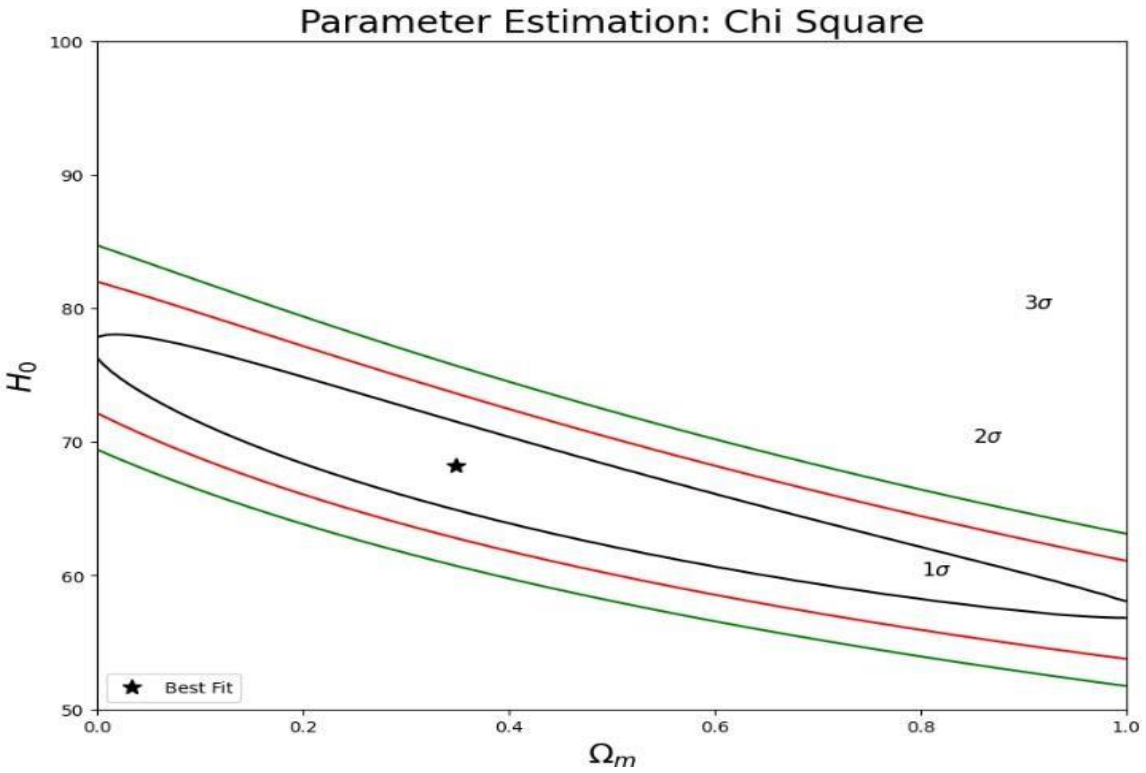

Fig. 11 Two parameter contours, here 1σ contour represents all those values of Ω_m and H_0 where $\chi^2 = \chi^2_{min} \pm 2.3$, for 2σ $\chi^2 = \chi^2_{min} \pm 6.18$ and for the 3σ it is $\chi^2 = \chi^2_{min} \pm 11.83$ [Appendix II – 4]

3. Markov Chain Monte Carlo

Markov Chain Monte Carlo popularly known as MCMC technique, is an algorithm used for systematic random sampling from high-dimensional random probability distributions. It uses Markov chains to perform Monte Carlo estimate. Markov property simply makes an assumption — "*the probability of jumping from one state to the next state depends only on the current state and not on the sequence of previous states that lead to this current state.*" The General Idea for the algorithm is to start with some random probability distribution and gradually move towards desired probability distribution

Generating Markov chains for multiple walkers for and we get the following chains for the parameters Ω_m and H_0

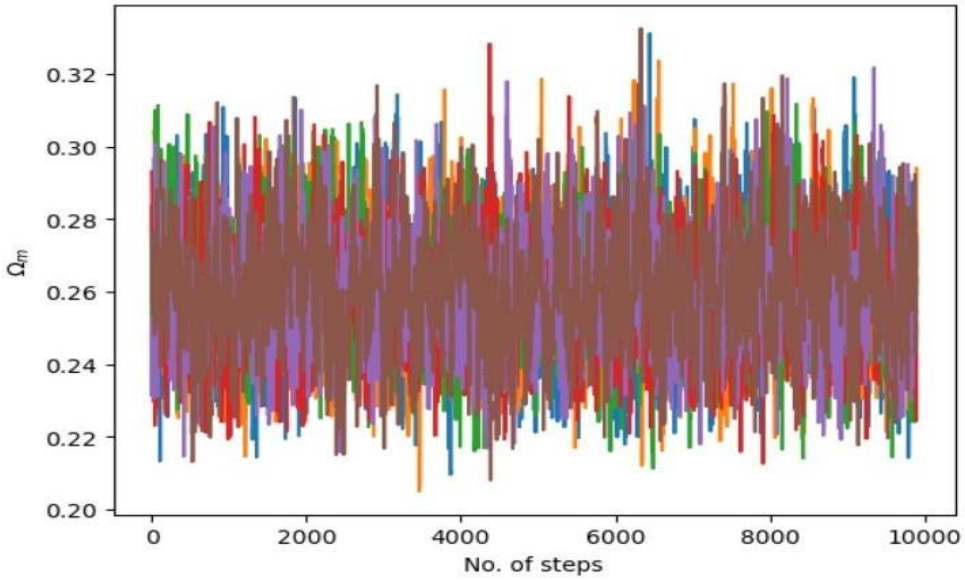

Fig. 12 Markov Chains generated for Ω_m for 6 walkers, and each walker for 10000 steps converges at 0.26 [Appendix II – 5]

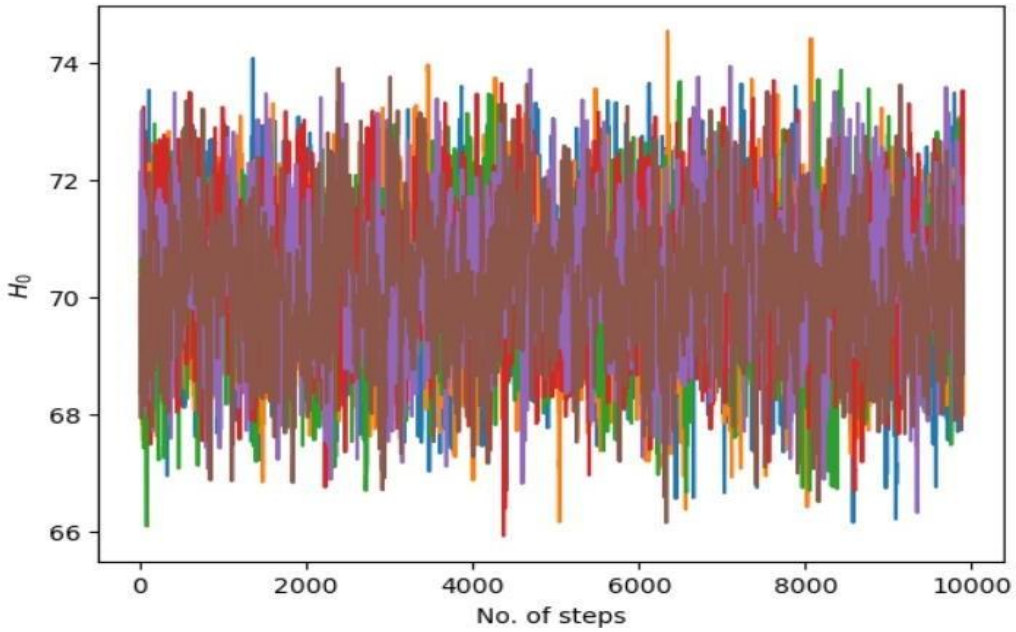

Fig. 13 Markov Chains generated for H_0 for 6 walkers, and each walker for 10000 steps converges at 70 [Appendix II – 5]

We can quote 68% confidence intervals of Ω_m *and* H_0 in a corner plot as shown -

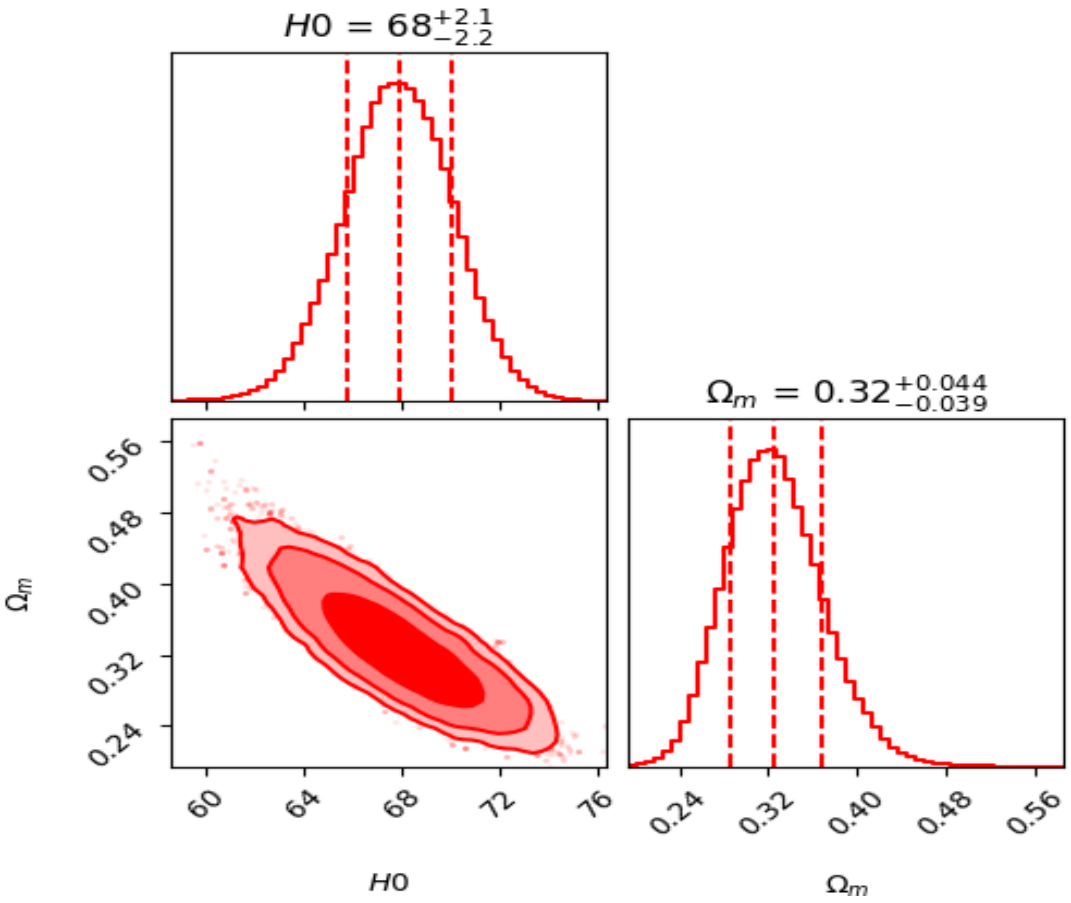

Fig. 14 1σ confidence intervals of Ω_m and H_0 [Appendix II – 5]

Appendix II

Codes for Appendix I

1. Monte Carlo π estimation –

```
importnumpy as np

importmatplotlib.pyplot as plt

# Number of points to generate

n = int(input("Enter the number of points you want to generate: "))

# Generate random points in the first quadrant [0, 1] x [0, 1]

x = np.random.uniform(0, 1, n)

y = np.random.uniform(0, 1, n)

# Check if points are inside the unit circle

inside_circle = (x**2 + y**2) <= 1

# Estimate pi (Area of unit circle in first quadrant is pi/4)

# The fraction of points inside the circle to the total points gives pi/4

pi_estimate = (np.sum(inside_circle) / n) * 4

# Print the estimate

print(f"Estimated value of pi: {pi_estimate}")

# Plotting the points

plt.figure(figsize=(8, 8))

plt.scatter(x[inside_circle], y[inside_circle], color='blue', s=1, label='Inside Circle')

plt.scatter(x[~inside_circle], y[~inside_circle], color='red', s=1, label='Outside Circle')

plt.title('Estimating Pi Using Monte Carlo Method (Quadrant)')

plt.xlabel('x')

plt.ylabel('y')

plt.legend()

plt.gca().set_aspect('equal',  adjustable='box')
```

```
plt.show()
```

2. Estimation of area under curve

```
importnumpy as np
importmatplotlib.pyplot as plt
# Defining the function
def f(x):
    returnnp.sin(x)
# Defining the intervals
a = 0
b = np.pi
# Finding the maximum value
def max(a, b):
    xv = np.linspace(a, b, 1000)
    maxidx = np.argmax(f(xv))
    max_x = xv[maxidx]
    max_y = f(max_x)
    returnmax_x, max_y
# Defining the number of darts
N = int(input("enter the number of points to be generated- "))
# Finding maximum x and y
max_x, max_y = max(a, b)
# Generating random points
xran = np.random.uniform(a, b, N)
yran = np.random.uniform(0, max_y, N)  # Use the correct max_y
# Counting points below the curve
nb = np.sum(yran<= f(xran))
```

```
# Rectangle area

rar = (b - a) * max_y

# Estimating

intg=(nb/N)*rar

#plotting the graph

x=np.linspace(a,b,500)

y=f(x)

plt.figure(figsize=(10,6))

plt.plot(x,y,label='f(x)',color="blue")

plt.fill_between(x,y,color="skyblue",alpha=0.4)

plt.scatter(xran,yran,color="red",s=1,label="random points")

plt.scatter(xran[yran<=f(xran)],yran[yran<=f(xran)],color="green",s=1,label="below curve")

plt.ylim(0,max_y*1.1)

plt.title("Monte Carlo Integration")

plt.xlabel("x")

plt.ylabel("y")

plt.legend()

plt.show()

print("estimated integral value is ", intg)

print("Maximum value of function is ",max_y)
```

3. Chi–Square Method (One parameter fitting)

```
importnumpy as np

importmatplotlib.pyplot as plt

fromscipy.optimize import minimize

# Data

z = np.array([0.07, 0.09, 0.12, 0.17, 0.1791, 0.1993, 0.2, 0.27, 0.28, 0.35, 0.3519, 0.3802, 0.4])

h_observed = np.array([69, 69, 68.6, 83, 75, 75, 72.9, 77, 88.8, 82.7, 83, 83, 95])
```

```
E = np.array([19.6, 12, 26.2, 8, 4, 5, 29.6, 14, 36.6, 8.4, 14, 13.5, 17])
h0 = 67.8  # Hubble constant
defmodel_function(z, Omega_m, h0):
    #Calculates the theoretical Hubble parameter (h) for a given redshift (z).
    return h0 * np.sqrt(Omega_m * (1 + z) ** 3 + (1 - Omega_m))
defchi_square(Omega_m):
   #Calculates the chi-square value for a given Omega_m.
    h_theoretical = model_function(z, Omega_m, h0)
    chi2 = np.sum(((h_observed - h_theoretical) / E) ** 2)
    return chi2
# Omega_m range (0 to 1 with 100 steps)
Omega_m_values = np.linspace(0, 1, 100)
# Calculate chi-square for each Omega_m value
chi_square_values = [chi_square(val) for val in Omega_m_values]
# Find the minimum chi-square value
min_chi_square = min(chi_square_values)
# Plot chi-square vsOmega_m
plt.figure(figsize=(10, 6))
plt.plot(Omega_m_values, chi_square_values, label='$\chi^2$ Curve')
plt.axhline(y=min_chi_square + 1, color='r', linestyle='--', label=r'1$\sigma$ Confidence Level')
plt.axhline(y=min_chi_square + 4, color='g', linestyle='--', label=r'2$\sigma$ Confidence Level')
plt.axhline(y=min_chi_square + 9, color='b', linestyle='--', label=r'3$\sigma$ Confidence Level')
plt.xlabel(r'$\Omega_m$', fontsize=14)
plt.ylabel(r'$\chi^2$', fontsize=14)
plt.title(r'$\chi^2$ vs. $\Omega_m$ (Hubble Constant Data)', fontsize=16)
plt.legend()
plt.grid(True)
plt.savefig("one_parameter",dpi = 200)
```

```
plt.show()
plt.grid(True)
```

4. Chi-square method (Two parameter contours plotting)

```
importnumpy as np
fromscipy.optimize import minimize
importmatplotlib.pyplot as plt
# Data (use your actual data here)
z = np.array([0.07, 0.09, 0.12, 0.17, 0.1791, 0.1993, 0.2, 0.27, 0.28, 0.35, 0.3519, 0.3802, 0.4])
h_observed = np.array([69, 69, 68.6, 83, 75, 75, 72.9, 77, 88.8, 82.7, 83, 83, 95])
E = np.array([19.6, 12, 26.2, 8, 4, 5, 29.6, 14, 36.6, 8.4, 14, 13.5, 17])
defmodel_function(z, params):
    Omega_m, h0 = params
    return h0 * np.sqrt(Omega_m * (1 + z)**3 + (1 - Omega_m))
defchi_square(params):
    h_theoretical = model_function(z, params)
    returnnp.sum(((h_observed - h_theoretical) / E)**2)
# Minimize chi-square function
initial_guess = [0.3, 67.8]
result = minimize(chi_square, initial_guess, method='Nelder-Mead')
print("Minimum chi-square value:", result.fun)
print("Optimal Omega_m:", result.x[0])
print("Optimal h0:", result.x[1])
# Calculate chi-square for a range of Omega_m and h0 values
Omega_m_range = np.linspace(0, 1, 100)
h0_range = np.linspace(50, 100, 100)
chi_square_values = np.zeros((100, 100))
```

```
for i, om in enumerate(Omega_m_range):
    for j, h0 in enumerate(h0_range):
        chi_square_values[i, j] = chi_square([om, h0])
# Plotting
plt.figure(figsize=(10, 8))
# Calculate delta chi-square values for confidence intervals
chi_min = np.min(chi_square_values)
delta_chi_1sigma = 2.30  # For 2D, 68.3% confidence
delta_chi_2sigma = 6.17  # For 2D, 95.4% confidence
delta_chi_3sigma = 11.8  # For 2D, 99.7% confidence
# Plot contours
plt.contour(Omega_m_range, h0_range, chi_square_values.T,
                    levels=[chi_min + delta_chi_1sigma, chi_min + delta_chi_2sigma, chi_min +
delta_chi_3sigma],
          colors=['black', 'red', 'green'], linestyles=['solid', 'solid', 'solid'])
plt.xlabel(r'$\Omega_m$', fontsize=18)
plt.ylabel(r'$H_0$', fontsize=18)
plt.title('Parameter Estimation: Chi Square', fontsize=20)
# Mark the best fit point
plt.plot(result.x[0], result.x[1], 'k*', markersize=10, label='Best Fit')
plt.legend(loc="lower left")
plt.xlim(0, 1)
plt.ylim(50, 100)
# Add labels for confidence intervals
plt.text(0.8, 60, r'1$\sigma$', fontsize=12)
plt.text(0.85, 70, r'2$\sigma$', fontsize=12)
plt.text(0.9, 80, r'3$\sigma$', fontsize=12)
plt.savefig("omega_m_h0_contours.png")
```

```
plt.show()
```

5. Markov Chain Monte Carlo

```
import emcee

import corner

import time

importnumpy as np

importscipy.optimize as op

importmatplotlib.pyplot as plt

fromnumpy import *

z=np.array([0.07  , 0.09  , 0.12  , 0.17  , 0.179 , 0.199 , 0.2   , 0.27  ,
      0.28  , 0.352 , 0.3802, 0.4   , 0.4004, 0.4247, 0.4497, 0.47  ,
      0.4783, 0.48  , 0.5929, 0.6797, 0.7812, 0.8754, 0.88  , 0.9   ,
      1.037 , 1.3   , 1.363 , 1.43  , 1.53  , 1.75  , 1.965 ]);

hz= np.array([ 69. ,  69. ,  68.6,  83. ,  75. ,  75. ,  72.9,  77. ,  88.8,
       83. ,  83. ,  95. ,  77. ,  87.1,  92.8,  89. ,  80.9,  97. ,
      104. ,  92. , 105. , 125. ,  90. , 117. , 154. , 168. , 160. ,
      177. , 140. , 202. , 186.5])

shz=np.array([19.6, 12. , 26.2,  8. ,  4. ,  5. , 29.6, 14. , 36.6, 14. , 13.5,
      17. , 10.2, 11.2, 12.9, 34. ,  9. , 62. , 13. ,  8. , 12. , 17. ,
      40. , 23. , 20. , 17. , 33.6, 18. , 14. , 40. , 50.4])

deflog_likelihood(z,hz,shz,h0,om):
  n=len(z)
  sum=0
  for i in range(0,n):
    sum= sum+ (hz[i]-h0*(om*(1+z[i])**3+(1-om))**0.5)**2/(shz[i])**2
  return -(sum)

def log_prior1(h0,om):
```

```
 if 50< h0 < 100 and 0<om< 1:
      return 0.0
 return -np.inf
deflog_posterior(theta):
 h0,om= theta
 lp=log_prior1(h0,om)
 func= log_likelihood(z,hz,shz,h0,om)+ log_prior1(h0,om)
 if not np.isfinite(lp):
  return -np.inf
 returnfunc
importscipy.optimize as optimize
def chi2(theta):
   h0, om = theta
   chi= -2 * log_posterior(theta)
   return chi
initial_guess = [70, 0.3]
result = optimize.minimize(chi2,initial_guess)
ndimension=2
no_of_walkers=6
no_of_steps_per_walker= 10000
initial_pos_of_walker = [result["x"] + 1e-4*np.random.randn(ndimension) for i in
range(no_of_walkers)]
print(initial_pos_of_walker)
sampler = emcee.EnsembleSampler(no_of_walkers, ndimension, log_posterior, a=2)
print(sampler)


sampler.run_mcmc(initial_pos_of_walker,no_of_steps_per_walker,rstate0=np.random.get_state(),p
rogress=True);
```

```
burn_in_steps=100
for i in range(0,no_of_walkers):
  a=sampler.chain[i,burn_in_steps:,0]
  l=len(a)
  n= list(range(l))
plt.plot(n,a)
plt.xlabel('No. of steps')
plt.ylabel('$H_{\\mathrm{0}}$')
plt.show()
for i in range(0,no_of_walkers):
  a=sampler.chain[i,burn_in_steps:,1]
  l=len(a)
  n= list(range(l))
plt.plot(n,a)
plt.xlabel('No. of steps')
plt.ylabel('$\\Omega_m$')
plt.show()
samples = sampler.chain[:, burn_in_steps:, :].reshape((-1, ndimension))
fig = corner.corner(samples,bins=50,labels=["$H0$","$\Omega_m$"],
                    color="red",
                    quantiles=[0.16, 0.5, 0.84],
                    plot_contours=True,
                    fill_contours=True,
                    levels=(0.68,0.95,0.99,),
                    plot_datapoints=True,
                    smooth=True, smooth1d=True,
                    title_fmt=".2g",
```

```
            show_titles=True,
            divergences=True)
plt.savefig("emcmc_result.png",dpi=200)
plt.show()
```

Codes for DESI BAO analysis

6. Surface plot

```
importnumpy as np
fromscipy.integrate import quad
importplotly.graph_objects as go
import emcee
import corner
importmatplotlib.pyplot as plt
# Constants
c = 299792.458  # Speed of light in km/s
# Data from the table
data = {
    "tracer": ["BGS", "LRG1", "LRG2", "LRG3+ELG1", "ELG2", "QSO", "Lya QSO"],
    "zeff": np.array([0.295, 0.510, 0.706, 0.930, 1.317, 1.491, 2.330]),
    "DM_rd": np.array([np.nan, 13.62, 16.85, 21.71, 27.79, np.nan, 39.71]),
    "DM_rd_err": np.array([np.nan, 0.25, 0.32, 0.28, 0.69, np.nan, 0.94]),
    "DH_rd": np.array([np.nan, 20.98, 20.08, 17.88, 13.82, np.nan, 8.52]),
    "DH_rd_err": np.array([np.nan, 0.61, 0.60, 0.35, 0.42, np.nan, 0.17]),
    "DV_rd": np.array([7.93, np.nan, np.nan, np.nan, np.nan, 26.07, np.nan]),
    "DV_rd_err": np.array([0.15, np.nan, np.nan, np.nan, np.nan, 0.67, np.nan])
}

# Hubble parameter in the Lambda CDM model
```

```
def E(z, omega_m):
omega_lambda = 1 - omega_m
returnnp.sqrt(omega_m * (1 + z)**3 + omega_lambda)
# Comoving angular diameter distance DM
def DM(z, omega_m, h0_rd):
integral, _ = quad(lambda z_prime: c / (h0_rd * E(z_prime, omega_m)), 0, z)
return integral
# Hubble distance DH
def DH(z, omega_m, h0_rd):
return c / (h0_rd * E(z, omega_m))
# Volume-averaged distance DV
def DV(z, omega_m, h0_rd):
dm = DM(z, omega_m, h0_rd)
return (((1 + z)**2 * dm**2 * c * z) / (h0_rd * E(z, omega_m)))**(1/3)
# Chi-squared calculation
defchi_squared(params, tracer_data):
omega_m, h0, rd = params
    h0_rd = h0 * rd
    chi2 = 0
zeff = tracer_data["zeff"]
  # Check each element individually for NaN
for i in range(len(zeff)):
if not np.isnan(tracer_data["DM_rd"][i]):
dm_rd_theoretical = DM(zeff[i], omega_m, h0_rd)
        chi2 += ((tracer_data["DM_rd"][i] - dm_rd_theoretical) / tracer_data["DM_rd_err"][i])**2


if not np.isnan(tracer_data["DH_rd"][i]):
```

```
dh_rd_theoretical = DH(zeff[i], omega_m, h0_rd)
        chi2 += ((tracer_data["DH_rd"][i] - dh_rd_theoretical) / tracer_data["DH_rd_err"][i])**2
if not np.isnan(tracer_data["DV_rd"][i]):
dv_rd_theoretical = DV(zeff[i], omega_m, h0_rd)
        chi2 += ((tracer_data["DV_rd"][i] - dv_rd_theoretical) / tracer_data["DV_rd_err"][i])**2
return chi2
# Log-likelihood for MCMC
deflog_likelihood(params, tracer_data):
omega_m, h0, rd = params
return -0.5 * chi_squared(params, tracer_data)
# Log-prior for MCMC
deflog_prior(params):
omega_m, h0, rd = params
if 0 <omega_m< 1 and 50 < h0 < 100 and 147.46 - 3*0.28 <rd< 147.46 + 3*0.28:
return -0.5 * ((rd - 147.46) / 0.28)**2
return -np.inf
# Log-posterior for MCMC
deflog_posterior(params, tracer_data):
lp = log_prior(params)
if not np.isfinite(lp):
return -np.inf
returnlp + log_likelihood(params, tracer_data)
# Grid for omega_m, h0, and rd
omega_m_values = np.linspace(0.1, 0.5, 50)
h0_values = np.linspace(50, 100, 50)
rd_values = np.linspace(147.46 - 3*0.28, 147.46 + 3*0.28, 50)
```

```
# Evaluate the log-posterior over the grid

log_posterior_grid = np.zeros((len(omega_m_values), len(h0_values), len(rd_values)))

for i, omega_m in enumerate(omega_m_values):

for j, h0 in enumerate(h0_values):

for k, rd in enumerate(rd_values):

log_posterior_grid[i, j, k] = log_posterior([omega_m, h0, rd], data)

# Marginalize over rd to get a 2D grid for omega_m and h0_rd

log_posterior_marginalized = np.max(log_posterior_grid, axis=2)

# Create the surface plot using Plotly

fig = go.Figure(data=[go.Surface(z=log_posterior_marginalized, x=omega_m_values, y=h0_values*rd_values)])

fig.update_traces(contours_z=dict(show=True, usecolormap=True,

highlightcolor="limegreen", project_z=True))

fig.update_layout(title='Log Posterior Surface (Marginalized over rd)', autosize=False,

scene_camera_eye=dict(x=1.87, y=0.88, z=-0.64),

width=800, height=800,

margin=dict(l=65, r=50, b=65, t=90))

fig.show()
```

7.

```
import numpy as np

fromscipy.integrate import quad

import emcee

import corner

importmatplotlib.pyplot as plt

# Constants

c = 299792.458  # Speed of light in km/s


# Data from the table

data = {
```

```
    "tracer": ["BGS", "LRG1", "LRG2", "LRG3+ELG1", "ELG2", "QSO", "Lya QSO"],
    "zeff": np.array([0.295, 0.510, 0.706, 0.930, 1.317, 1.491, 2.330]),
    "DM_rd": np.array([np.nan, 13.62, 16.85, 21.71, 27.79, np.nan, 39.71]),
    "DM_rd_err": np.array([np.nan, 0.25, 0.32, 0.28, 0.69, np.nan, 0.94]),
    "DH_rd": np.array([np.nan, 20.98, 20.08, 17.88, 13.82, np.nan, 8.52]),
    "DH_rd_err": np.array([np.nan, 0.61, 0.60, 0.35, 0.42, np.nan, 0.17]),
    "DV_rd": np.array([7.93, np.nan, np.nan, np.nan, np.nan, 26.07, np.nan]),
    "DV_rd_err": np.array([0.15, np.nan, np.nan, np.nan, np.nan, 0.67, np.nan])
}
# Hubble parameter in the Lambda CDM model
def E(z, omega_m):
omega_lambda = 1 - omega_m
returnnp.sqrt(omega_m * (1 + z)**3 + omega_lambda)
# Comoving angular diameter distance DM
def DM(z, omega_m, h0_rd):
integral, _ = quad(lambda z_prime: c / (h0_rd * E(z_prime, omega_m)), 0, z)
return integral
# Hubble distance DH
def DH(z, omega_m, h0_rd):
return c / (h0_rd * E(z, omega_m))
# Volume-averaged distance DV
def DV(z, omega_m, h0_rd):
dm = DM(z, omega_m, h0_rd)
return (((1 + z)**2 * dm**2 * c * z) / (h0_rd * E(z, omega_m)))**(1/3)

# Chi-squared calculation
defchi_squared(params, tracer_data):
```

```
omega_m, h0, rd = params
    h0_rd = h0 * rd
    chi2 = 0
zeff = tracer_data["zeff"]
    # Check each element individually for NaN
for i in range(len(zeff)):
if not np.isnan(tracer_data["DM_rd"][i]):
dm_rd_theoretical = DM(zeff[i], omega_m, h0_rd)
        chi2 += ((tracer_data["DM_rd"][i] - dm_rd_theoretical) / tracer_data["DM_rd_err"][i])**2

if not np.isnan(tracer_data["DH_rd"][i]):
dh_rd_theoretical = DH(zeff[i], omega_m, h0_rd)
        chi2 += ((tracer_data["DH_rd"][i] - dh_rd_theoretical) / tracer_data["DH_rd_err"][i])**2

if not np.isnan(tracer_data["DV_rd"][i]):
dv_rd_theoretical = DV(zeff[i], omega_m, h0_rd)
        chi2 += ((tracer_data["DV_rd"][i] - dv_rd_theoretical) / tracer_data["DV_rd_err"][i])**2
return chi2
# Log-likelihood for MCMC
deflog_likelihood(params, tracer_data):
omega_m, h0, rd = params
return -0.5 * chi_squared(params, tracer_data)
# Log-prior for MCMC
deflog_prior(params):
omega_m, h0, rd = params
if 0 <omega_m< 1 and 50 < h0 < 100:
return -0.5 * ((rd - 147.5) / 0.28)**2
```

```
return -np.inf
# Log-posterior for MCMC
deflog_posterior(params, tracer_data):
lp = log_prior(params)
if not np.isfinite(lp):
return -np.inf
returnlp + log_likelihood(params, tracer_data)
# Initial guess for omega_m, h0, and rd
initial_guess = [0.3, 75, 147.5]
# Perform the fit for each tracer using MCMC
ndim = 3
nwalkers = 10
nsteps = 5000
initial_positions = [initial_guess + 1e-4 * np.random.randn(ndim) for _ in range(nwalkers)]
sampler = emcee.EnsembleSampler(nwalkers, ndim, log_posterior, args=(data,))
sampler.run_mcmc(initial_positions, nsteps, progress=True)
# Extract the samples
samples = sampler.get_chain(flat=True)
# Calculate h0_rd
h0_rd_samples = samples[:, 1] * samples[:, 2]
omega_m_samples = samples[:, 0]
# Create new samples array for corner plot
new_samples = np.vstack([omega_m_samples, h0_rd_samples]).T
# Plot the corner plot
fig = corner.corner(new_samples, bins=50, labels=["$\Omega_m$", "$H_0 r_d$"],
color="blue",
quantiles=[0.16, 0.5, 0.84],
```

```
plot_contours=True,
fill_contours=True,
levels=(0.68, 0.95, 0.99,),
plot_datapoints=True,
smooth=True, smooth1d=True,
title_fmt=".2f",  # Change to use 2 decimal places
show_titles=True,
title_kwargs={"fontsize": 12})
plt.show()
```

8.
```
import numpy as np
fromscipy.integrate import quad
fromscipy.optimize import minimize
# Constants
c = 299792.458  # Speed of light in km/s
# Data from the table
data = {
    "tracer": ["BGS", "LRG1", "LRG2", "LRG3+ELG1", "ELG2", "QSO", "Lya QSO"],
    "zeff": np.array([0.295, 0.510, 0.706, 0.930, 1.317, 1.491, 2.330]),
    "DM_rd": np.array([np.nan, 13.62, 16.85, 21.71, 27.79, np.nan, 39.71]),
    "DM_rd_err": np.array([np.nan, 0.25, 0.32, 0.28, 0.69, np.nan, 0.94]),
    "DH_rd": np.array([np.nan, 20.98, 20.08, 17.88, 13.82, np.nan, 8.52]),
    "DH_rd_err": np.array([np.nan, 0.61, 0.60, 0.35, 0.42, np.nan, 0.17]),
    "DV_rd": np.array([7.93, np.nan, np.nan, np.nan, np.nan, 26.07, np.nan]),
    "DV_rd_err": np.array([0.15, np.nan, np.nan, np.nan, np.nan, 0.67, np.nan]),
    "Ntracer": np.array([300017, 506905, 771875, 1876164, 1415687, 856652, 709565])
}
# Hubble parameter in the Lambda CDM model
```

```
def E(z, omega_m):

omega_lambda = 1 - omega_m

returnnp.sqrt(omega_m * (1 + z)**3 + omega_lambda)

# Comoving angular diameter distance DM

def DM(z, omega_m, h0rd):

integral, _ = quad(lambda z_prime: c / (h0rd * E(z_prime, omega_m)), 0, z)

return integral

# Hubble distance DH

def DH(z, omega_m, h0rd):

return c / (h0rd * E(z, omega_m))

# Volume-averaged distance DV

def DV(z, omega_m, h0rd):

dm = DM(z, omega_m, h0rd)

return (( (1 + z)*2 * dm**2 * c * z / (h0rd * E(z, omega_m)) ) ** (1/3))

# Chi-squared calculation for BAO

defchi_squared_BAO(params,  tracer_data):

omega_m, h0rd = params

   chi2_B1 = 0

   chi2_B2 = 0

   chi2_B3 = 0

zeff = tracer_data["zeff"]

Ntracer = tracer_data["Ntracer"]

if not np.isnan(tracer_data["DV_rd"]):

dv_rd_theoretical = DV(zeff, omega_m, h0rd)

      chi2_B1 += Ntracer * ((tracer_data["DV_rd"] - dv_rd_theoretical) /
tracer_data["DV_rd_err"])**2

if not np.isnan(tracer_data["DM_rd"]):

dm_rd_theoretical = DM(zeff, omega_m, h0rd)
```

```
        chi2_B2 += Ntracer * ((tracer_data["DM_rd"] - dm_rd_theoretical) /
tracer_data["DM_rd_err"])**2

if not np.isnan(tracer_data["DH_rd"]):

dh_rd_theoretical = DH(zeff, omega_m, h0rd)

        chi2_B3 += Ntracer * ((tracer_data["DH_rd"] - dh_rd_theoretical) /
tracer_data["DH_rd_err"])**2

    chi2_BAO = chi2_B1 + chi2_B2 + chi2_B3

return chi2_BAO

# Perform the fit for each tracer

initial_guess = [0.3, 10000]  # Initial guess for omega_m and h0rd

best_fit_params = []

chi2_min_values = []

for i in range(len(data["tracer"])):

tracer_data = {

        "zeff": data["zeff"][i],

        "DM_rd": data["DM_rd"][i],

        "DM_rd_err": data["DM_rd_err"][i],

        "DH_rd": data["DH_rd"][i],

        "DH_rd_err": data["DH_rd_err"][i],

        "DV_rd": data["DV_rd"][i],

        "DV_rd_err": data["DV_rd_err"][i],

        "Ntracer": data["Ntracer"][i]

    }

    # Minimize the chi-squared function for this tracer

result = minimize(chi_squared_BAO, initial_guess, args=(tracer_data,), bounds=[(0, 1), (5000,
20000)])

best_fit_params.append(result.x)

    chi2_min_values.append(result.fun)

# Output the best-fit parameters and chi-squared values for each tracer
```

```
for i in range(len(data["tracer"])):

print(f"Tracer: {data['tracer'][i]}")

print(f"Best-fit Omega_m: {best_fit_params[i][0]}")

print(f"Best-fit H0 * rd: {best_fit_params[i][1]}")

print(f"Minimum chi-squared: {chi2_min_values[i]:.2e}\n")
```

9. iimport numpy as np

importmatplotlib.pyplot as plt

fromastropy.cosmology import Planck15

```
# Data

omega_m = np.array([0.66, 0.23, 0.27, 0.34, 0.37])

zeff = np.array([0.51, 0.71, 0.93, 1.32, 2.33])

error = np.array([[0.16, 0.15], [0.07, 0.06], [0.05, 0.04], [0.08, 0.07], [0.07, 0.06]])

# Planck15 value for omega_m

planck_omega_m = Planck15.Om0

print(planck_omega_m)

# Define sigma intervals (assuming they are provided or approximated)

sigma_1 = 0.31  # 1 sigma

sigma_2 = 2 * sigma_1  # 2 sigma

sigma_3 = 3 * sigma_1  # 3 sigma

# Separate the errors into positive and negative

yerr = [error[:, 0], error[:, 1]]

# Plot

plt.errorbar(zeff, omega_m, yerr=yerr, fmt='o', capsize=5)

# Plot the Planck Omega_m value and sigma intervals

plt.axhline(y=planck_omega_m, color='r', linestyle='--', label=f'Planck $\\Omega_m$ =
{planck_omega_m:.2f}')

plt.axhline(y=planck_omega_m + sigma_1, color='g', linestyle='-', label='$1 \\sigma$')

plt.axhline(y=planck_omega_m - sigma_1, color='g', linestyle='-')
```

```
plt.xlabel('$z_{\\mathrm{eff}}$')

plt.ylabel('$\\Omega_m$')

#plt.title('$\\Omega_m$ vs. $z_{\\mathrm{eff}}$ with Error Bars')

plt.legend(loc='best')

plt.show()
```